❶ The OpenTelemetry version

❷ The OpenTelemetry agent instrumenting the Java code dynamically via bytecode

Besides instrumenting the Java code to capture traces, the OpenTelemetry Java Agent also integrates with SLF4J (and its implementation). It provides trace and span identifiers as contextual information that can be injected into log messages through the MDC abstraction provided by SLF4J. That makes it extremely simple to navigate from log messages to traces and vice versa, achieving better visibility into the application than querying the telemetry in isolation.

Let's expand on the default log format used by Spring Boot and add the following contextual information:

- Application name (value from the `spring.application.name` property we configured for all applications)
- Trace identifier (value from the `trace_id` field populated by the OpenTelemetry agent, when enabled)
- Span identifier (value from the `span_id` field populated by the OpenTelemetry agent, when enabled)

In your Catalog Service project, open the application.yml file, and add the three new pieces of information next to the log level (represented by `%5p`) following the Logback syntax. This is the same format used by Spring Cloud Sleuth.

Listing 13.21 Adding contextual information to logs, next to the level field

```
logging:
  pattern:
    level: "%5p [${spring.application.name},%X{trace_id},%X{span_id}]"    ❶
```